

# Design Documentation

## Subgroup 3: Devices

Revision History..... 1

Design item List..... 2

Design Item Descriptions..... 2

    D1. High-Level Architecture (Device → Gateway → Server → UI)..... 2

    D2. Arduino Device Component Design..... 4

    D3. Serial Communication Protocol..... 4

    D4. Interaction Flow – UI Command to Device..... 6

    D5 Fault Handling & Safe Behaviour..... 8

## Subgroup 3: Devices

### Revision History

| Date       | Version | Description                                                                        | Author                                                         |
|------------|---------|------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 2026-02-19 | 1.0     | Initial design document                                                            | Patrick Nordahl                                                |
| 2026-03-10 | 1.1     | Minor terminology refinement for clarity and consistency                           | Patrick Nordahl                                                |
| 2026-04-14 | 1.2     | Add refactored Gateway architecture                                                | Patrick Nordahl                                                |
| 2026-05-08 | 1.3     | Refactored Arduino architecture with Observer Pattern; updated Gateway to Node.js. | Mustafa Al-Bayati<br>Ryad Hazin<br>Tony Nguyen<br>Sergej Macut |

# Design item List

| Requirement Name                                                          | Priority  |
|---------------------------------------------------------------------------|-----------|
| D1. High-Level Architecture (Device → Gateway → Server → UI)              | Essential |
| D2. Arduino Device Component Design                                       | Essential |
| D3. Serial Communication Protocol (Message types + structured format)     | Essential |
| D4. Interaction Flow UI command to Devices                                | Essential |
| D5. Fault Handling & Safe State Behavior (Communication failure strategy) | Essential |

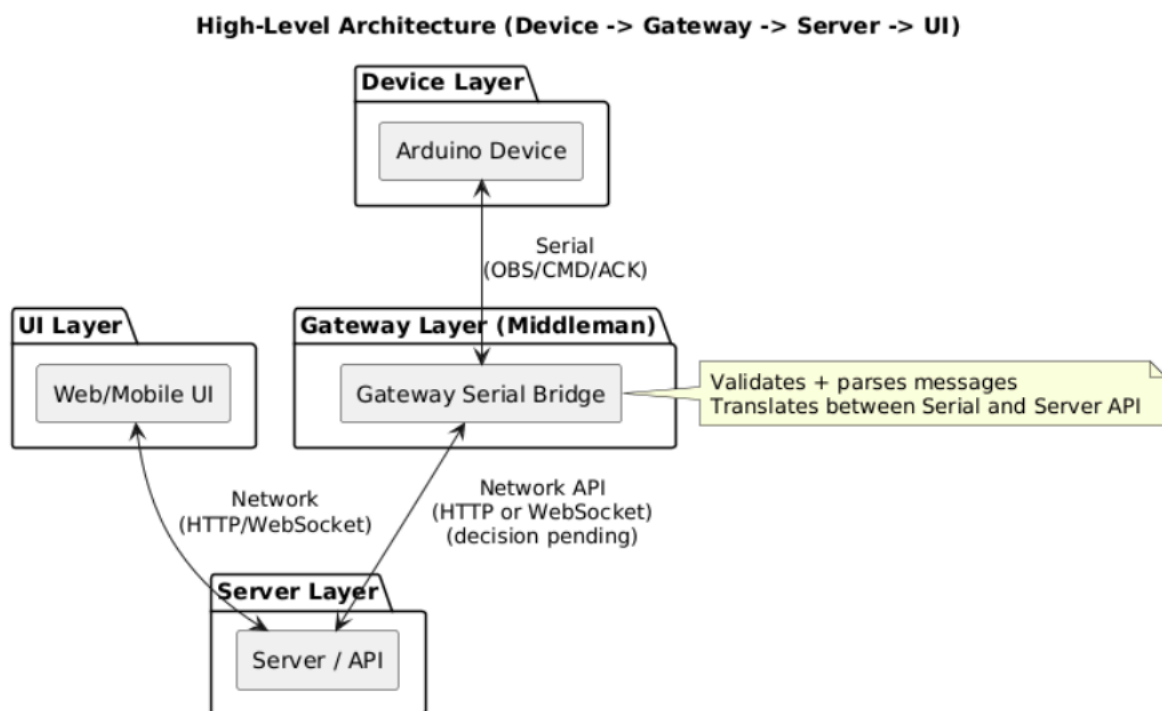
## Design Item Descriptions

### D1. High-Level Architecture (Device → Gateway → Server → UI)

**Description:** The system follows a layered architecture designed for modularity and asynchronous data flow. Communication is routed through a dedicated **Gateway**, which has been refactored from a Python-based script to a **Node.js (JavaScript)** environment. This change leverages Node's non-blocking I/O to handle simultaneous Serial and WebSocket communication more efficiently.

**The architecture consists of four logical layers:**

1. **Device Layer (Arduino):** Handles real-time sensor acquisition and actuator control. It utilizes a modular C++ structure with dedicated `.h` and `.cpp` files for each component.
2. **Gateway Layer (Node.js):** Acts as the middleman. It parses structured Serial messages (OBS/ACK), validates the data, and forwards it to the Server Layer via HTTP/WebSockets.
3. **Server Layer:** Centralizes system state and manages command distribution to clients.
4. **User Interface Layer:** Provides the dashboard for real-time monitoring and manual overrides.



**Figure 1.** Architecture/Component Diagram (D1)

This design ensures separation of concerns and supports interoperability through the middleman architecture.

## D2. Arduino Device Component Design

Description: The Arduino firmware has been refactored into a modular, object-oriented structure to ensure scalability and maintainability. Each hardware component (e.g., `DoorDevice`, `GasSafety`, `LCDManager`) is encapsulated in its own class across separate header (`.h`) and source (`.cpp`) files.

Key Architecture: Observer Design Pattern To reduce coupling between sensors and actuators, the system implements the Observer Design Pattern:

- Subjects (Sensors/Logic): Components like `GasSafety` and `SteamSensor` act as Subjects. They monitor thresholds and "notify" a list of subscribers when an event occurs.
- Observers (Actuators): Devices like `Fan`, `Buzzer`, `Door`, and `Window` implement an `Observer` interface. They wait for notifications and react independently when triggered.

Modular Components:

- Core Logic (`Observer.h`, `Subject.h`): Provides the base classes for the event-driven system.
- Sensors & Safety: Encapsulates the logic for threshold checking and state notification.
- Devices: Individual classes for actuators that handle physical hardware changes (PWM, digital writes).
- LCDManager: A dedicated display controller that aggregates "Active States" from the system to provide real-time user feedback.

## D3. Serial Communication Protocol

**Description:**

Structured Serial communication is used between the Arduino device and the gateway to ensure reliable data exchange.

Three conceptual message types are defined:

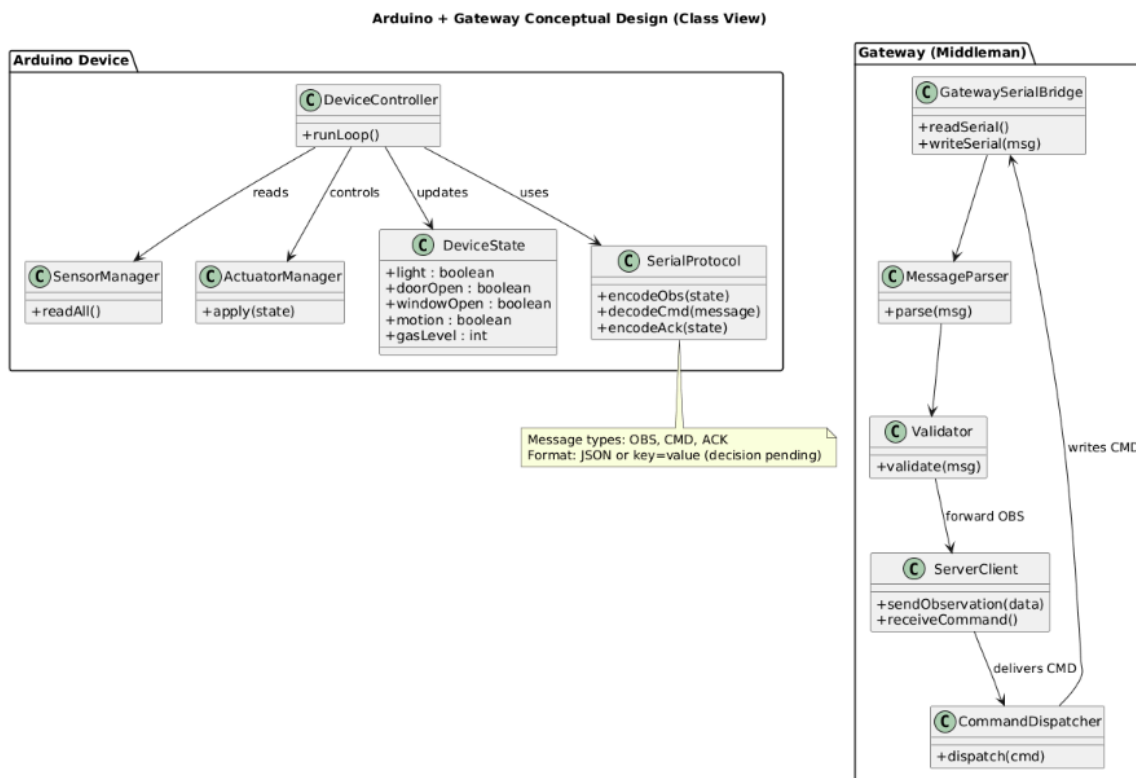
- OBS (Observation) – Sent from device to gateway (sensor values + state).
- CMD (Command) – Sent from gateway to device (state change request).
- ACK (Acknowledgement) – Sent from device to confirm applied command.

Example key-value format:

- obs;gas=312;motion=1;light=0
- cmd;light=1
- ack;light=1

Final message format (JSON or key-value) remains an open design decision.

Structured messaging reduces integration risks and supports validation.



**Figure 2.** Class diagram (D2+D3)

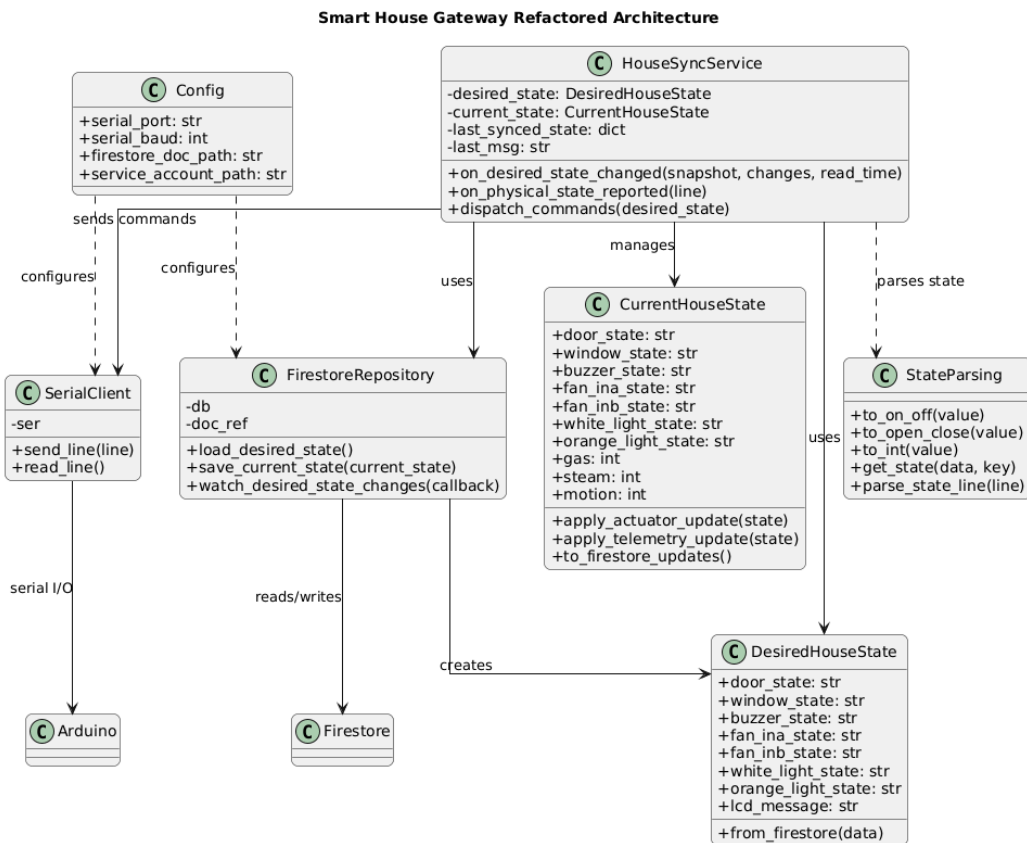


Figure 3. Refactored Gateway Architecture.

## D4. Interaction Flow – UI Command to Device

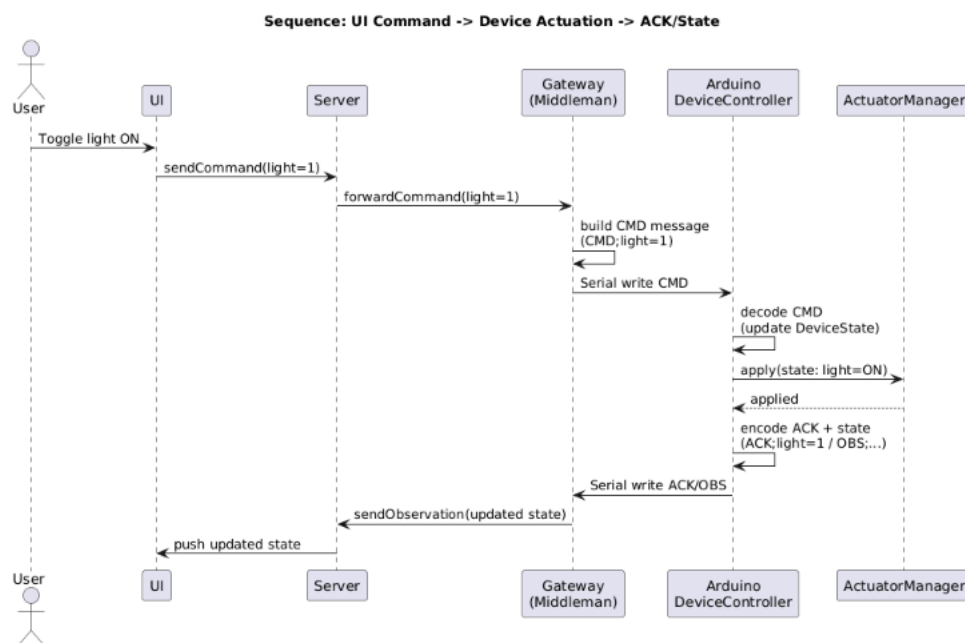
**Description:** The interaction flow describes how the system synchronizes state between the high-level UI and the low-level hardware using an event-driven approach.

### Scenario 1: Automated Event (Sensor Trigger)

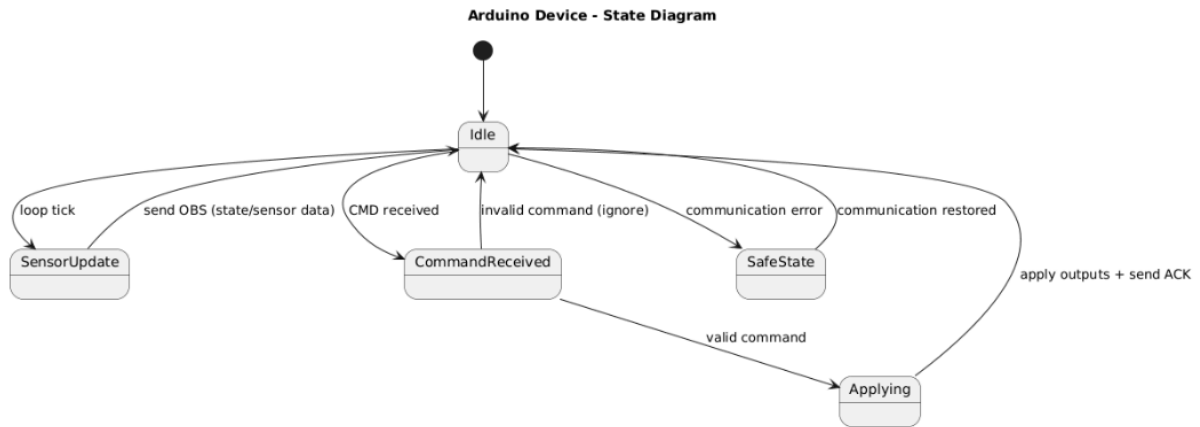
1. **Detection:** **SteamSensor** detects moisture (Rain).
2. **Notification:** The sensor (Subject) calls its **notify()** method.
3. **Update:** All attached Observers (**DoorDevice**, **WindowDevice**) receive the update and trigger their "Close" logic.
4. **Feedback:** The Arduino sends an **OBS** (Observation) or **ACK** message through the **Node.js Gateway** to update the UI state.

### Scenario 2: Manual Command (UI Toggle)

1. **UI:** User clicks "Toggle Light."
2. **Gateway:** The Node.js gateway receives the command from the server and writes a `CMD;light=1` string to the Serial port.
3. **Arduino:** The `CommandHandler` decodes the message and instructs the `ActuatorManager` to apply the state change.
4. **Verification:** The system sends an `ACK` back to ensure the UI reflects the actual hardware state.



**Figure 3.** Sequence Diagram (D4)



**Figure 4.** State Diagram (Arduino Device)

This flow ensures end-to-end synchronization between UI, server and device.

## D5 Fault Handling & Safe Behaviour

### Description:

To improve reliability, the device must handle communication failures gracefully.

If Serial communication is interrupted or corrupted:

- The Arduino continues operating in its last known safe state.
- Incoming commands must be validated before actuation.
- Invalid messages are ignored.
- The gateway is responsible for dropping corrupted data and logging errors.

This design reduces the impact of communication instability and addresses identified integration and reliability risks.